Real-Time Robotic Arm Controller with Priority Management

Background and overview

In modern industrial automation systems, robotic arms must respond to multiple concurrent tasks with strict timing and safety constraints. These systems often involve shared resources accessed by tasks of different priorities, which can lead to priority inversion, a critical issue in real-time systems.

This project focuses on designing and implementing a multi-threaded real-time simulation of a robotic arm controller in Java. Students will explore:

- Thread scheduling and priorities
- Resource sharing and synchronization
- Priority inversion and its consequences
- Priority inheritance and priority ceiling protocols

System description

You are required to simulate a robotic arm system consisting of three concurrent real-time tasks (threads) that share access to a critical resource.

2.1 System Components

The system includes:

A. Shared Resource

- MotorController
  - Simulates control of the robotic arm motor
  - Only one thread can access it at a time (critical section)

B. Real-time Threads and Responsibilities

Thread Name	Priority	Function
Safety Monitor	High	Detects emergency conditions and stops the arm
Motion Planner	Medium	Sends movement commands to the motor
Logger	Low	Records system activity

C. Key Behavior:

- All Real-time threads must access the shared MotorController
- Access must be synchronized
- Threads execute repeatedly to simulate real-time operation

Tasks

Task 1: Basic Multi-threaded Implementation

- Implement the three Real-time threads as independent Java classes
- Assign appropriate priorities using Java thread priority settings
- Implement the shared MotorController resource
- Ensure all threads repeatedly execute and attempt to access the resource
- Display clear console output showing:
    - Thread execution
    - Resource access
    - Timestamps

Task 2: Synchronization

- Implement mutual exclusion for the shared resource using:
- Ensure:
    - No race conditions occur
    - Only one thread accesses the resource at a time

Task 3: Priority Inversion Demonstration

- Create a controlled execution scenario that demonstrates **priority inversion**:
    - The low-priority Real-time thread acquires the resource
    - The high-priority Real-time thread attempts to access it and becomes blocked
    - The medium-priority Real-time thread executes and delays the release of the resource
- Your implementation must:
    - Clearly demonstrate the inversion behavior
    - Log execution order and blocking events
    - Measure and display the waiting time of the high-priority thread

Task 4: Priority Inheritance Protocol

- Modify your system to implement **priority inheritance**, such that:
    - When a high-priority thread is blocked by a lower-priority thread,
    - The lower-priority thread temporarily inherits the higher priority
- Since Java does not natively support this mechanism, simulate it using:
    - Dynamic adjustment of thread priorities during execution
- Your implementation must:
    - Reduce or eliminate the delay observed in Task 3

- Log priority changes and execution flow

Task 5: Priority Ceiling Protocol

- Implement the **priority ceiling protocol** for the shared resource:
  - Assign a ceiling priority to the MotorController
  - Any thread accessing the resource executes at this ceiling priority
- Your implementation must:
  - Prevent priority inversion from occurring
  - Ensure predictable execution behavior
  - Log when ceiling priority is applied and restored

Task 6: Performance Evaluation

- Evaluate and compare system performance for the following cases:
  1. Without any priority management (baseline)
  2. With priority inheritance
  3. With priority ceiling
- Measure and report:
  - Waiting time of the high-priority thread
  - Response time for critical operations
  - Any observed delays or improvements
- Conduct multiple runs and report average values
- Present results:
  - Numerically
  - Graphically (charts)

Implementation Requirements

- Use **standard Java and JamaicaVM APIs only**
- No third-party libraries are allowed
- Code must be modular and well-documented

What to Submit:

Code

- Complete Java implementation
- Clearly structured and commented

Report (Maximum 3–5 pages)

The report must include:

- Explanation of system design and implementation
- Description of how synchronization is achieved
- Demonstration of:
    - Priority inversion
    - Priority inheritance
    - Priority ceiling
- Performance evaluation with:
    - Tables
    - Graphs
- Critical discussion of results

Assessment Criteria

- Correctness and completeness of implementation
- Proper demonstration of real-time concepts
- Quality of performance evaluation and discussion
- Code structure and readability
- Clarity and professionalism of the report